Is the leak-free network running DD?

Causal interventions, not probe scores

YES -- it runs DD's inference, rule and guard.

DD = Direct Deduction, the classical 'naked single' solver the model is trained to imitate. It does two things at each step. A RULE: eliminate the colours a node's known neighbours already use. A GUARD: emit that node only when one colour is left. One-token counterfactuals test the two separately -- edit the forcing neighbour so that DD's answer changes (rule), or so that the node stops being determined at all (guard).

model (free-generation solve_rate)	RULE on given hints	RULE on its OWN facts	GUARD withholds
leak-free 1.0000	1.000	0.980	1.000
leak-free 0.8540	1.000	0.980	0.824
leaked 0.9960	0.388	0.008	0.992

Chance on the rule is 0.25. Both leak-free models apply the rule and chain it; only the one that solves the task guards on every case, and its 0.854 twin fails 0.176 of them (paired, p = 1e-13) -- the same order as that twin's 0.146 solve_rate deficit. The leaked model is the mirror image: it guards fine, and never learned the rule at all.

Prepared 2026-10-29. One task configuration (N=24 nodes, degree 5, q=4 colours) and one checkpoint per model -- no seed replicates, so every leak-free vs leaked contrast is also two separate training runs. Jobs 50075339 (round 0), 50077245 (chained), 50082120 (withholding).

The task, the algorithm, and the two kinds of model

THE TASK

Colour a random regular graph: N = 24 nodes, every node of degree 5, using q = 4 colours so that no edge joins two nodes of the same colour. Some nodes are given as HINTS (15 of 24 here); the rest must be deduced.

Built from a planted colouring by removing hints one at a time and keeping a removal only if DD can still recover everything -- so every instance is solvable by DD alone, and instances differ only in how deep the deduction goes.

DD, THE ALGORITHM THE MODEL IS TRAINED TO IMITATE

Direct Deduction, the 'naked single' rule: a node becomes determined one round after its known neighbours show all q-1 = 3 colours other than its own. Sweep the nodes, emit the determined ones, repeat. The number of rounds an instance needs is its reasoning depth T -- 1 to 6 at this density; the example on the next two pages needs 2.

THE MODEL

A 12-layer decoder-only transformer (d_model 256, 4 heads per layer, MLP 1024), trained from scratch with next-token cross-entropy on DD's own traces: graph, hints, then round by round the nodes DD determines and the colours it gives them. Scored by FREE GENERATION -- solve_rate is the fraction of 1500 held-out instances it colours completely and correctly, with no teacher forcing.

THE LEAK, AND WHY THERE ARE TWO KINDS OF MODEL

The generator listed each node's neighbours sorted by the neighbour's true colour, so a neighbour's POSITION in the adjacency row encoded its colour and 61-67% of nodes could be read off with no deduction at all. Permuting the rows dropped the old record model from 0.9975 to 0.1450.

'leaked'	trained on the sorted rows. Scores 0.9960 and needs no reasoning to do it.
'leak-free'	trained with the rows permuted. Two of them, 1.0000 and 0.8540, differing only by budget.

THE QUESTION: the leak-free 1.0000 model solves a task that cannot be done without deduction. Is it deducing, or doing something else that works here? Every number from here on is CAUSAL: edit one input token, ask whether the output follows DD's answer to the EDITED problem.

Why not probes: an earlier round read the model's own 'which node is next' features with a trained probe and got 0.545 of nodes right, while a hand rule that does no deduction at all -- 'the unknown node with the fewest unknown neighbours' -- got 0.632 on the same cases.

What the model actually sees

Every token is one categorical id into a 34-symbol vocabulary (N=24, q=4, degree=5). No text, no digits-as-characters.

VOCABULARY

0-5	`BEGIN_HINTS BEGIN_GRAPH BEGIN_REASONING` `BEGIN_ROUND SOLVED UNRESOLVED` *section + round markers*
6-9	`COLOR_0 COLOR_1 COLOR_2 COLOR_3` *the q colours*
10-33	`N0 N1 ... N23` *one id per node, no structure in the id*

GRAMMAR

`BEGIN_HINTS`	`(NODE COLOR)*`
`BEGIN_GRAPH`	`(NODE (NODE){degree})*`
`BEGIN_REASONING`	`(BEGIN_ROUND (NODE COLOR)+)*`
`SOLVED`	

The graph section is a neighbour table, not an image: node, then its degree neighbours. THE ORDER WITHIN A ROW IS THE LEAK.

THIS INSTANCE, 198 TOKENS (seed 50000000004)

HINTS	`BEGIN_HINTS N2 COLOR_2 N3 COLOR_2 N5 COLOR_2 N8 COLOR_3 N10 COLOR_0 N11 COLOR_1 ... (31 tokens)`
GRAPH	`BEGIN_GRAPH N0 N16 N5 N18 N13 N14 N1 N8 N13 N2 N11 N19 ... (145 tokens)`
REASONING	`BEGIN_REASONING BEGIN_ROUND N0 COLOR_1 N1 COLOR_0 N4 COLOR_0 N6 COLOR_1 N9 COLOR_1 N12 ... (22 tokens)`

Loss is on the REASONING section only: 2 rounds, 9 deductions, 18 target tokens. Everything before BEGIN_REASONING is the problem statement.

One real reasoning trace

15 hints of 24 nodes, then 2 rounds of simultaneous deduction. Hints: N2=2 N3=2 N5=2 N8=3 N10=0 N11=1 N13=3 N14=0 ...

THE TRACE, AS EMITTED

```
BEGIN_ROUND          N0 COLOR_1 N1 COLOR_0 N4 COLOR_0 N6 COLOR_1 ...
  8 deduced
BEGIN_ROUND          N7 COLOR_2
  1 deduced
SOLVED
```

WHY ONE DEDUCTION FIRES

node N0, neighbours [16, 5, 18, 13, 14]

colours known before this round:

```
[0, 2, 3]  (3 = q-1)
```

only colour left: 1

```
so the trace emits  N0 COLOR_1
```

The model is never shown this reasoning -- only the node/colour pair.

A round deduces every node whose neighbours already cover q-1 colours, all at once. Reasoning depth T is the number of rounds: a deduction in round 3 depends on ones made in rounds 1 and 2, which is the serial computation that network depth has to supply.

The intervention edits ONE colour token in this section (the chained test) or in the hints (the round-0 test).

T=2 rounds, the median of 40 draws from this config (range 1-6); seed 50000000004, shuffle_adj=True, shuffle_events=False; evidence from sudoku_tokens.evidence_rounds (pre-round state, as DD saw it)

The intervention: one token, two different correct answers

```
DD forces node v because its known neighbours already use q-1 = 3 of the 4 colours.
Exactly one of them, the neighbour w, holds colour a. So v must take the one colour
left over: c. Recolour w to c, and now a is the colour nobody holds, so DD says a.

   clean     ... BEGIN_ROUND  v  ->  [c]        w holds a, so c is v's only option
   corrupt   ... BEGIN_ROUND  v  ->  [a]        w recoloured to c, so a is now free

The ONLY difference between the two prompts is w's one colour token: build_pair
asserts (clean != corrupt).sum() == 1, so that is enforced, not assumed. Scored as
'moved to DD's new answer' -- did the prediction at v become a? Not 'is it still
accurate': the correct answer CHANGES, so a memorised output scores zero. Because the
model must pick one of q = 4 colours, chance on this test is 0.25.

Two versions, differing only in WHERE the edited token sits:

   ROUND 0    w is a HINT, handed to the model in the prompt. Tests whether the
              model applies the elimination rule to facts it was GIVEN.

   CHAINED    w was DEDUCED by the model itself in an earlier round, so the edited
              token sits in the REASONING section. Tests elimination against a fact
              the model produced -- propagation, not one rule application.
```

One weak direction, stated up front: editing an emitted deduction makes the prefix inconsistent, so a model that DETECTS the edit and keeps its own answer scores as 'does not chain'. Those cases are counted separately, in the table's next column.

The converse intervention: break the forcing, and v should vanish

Both tests on the previous page SELECT a node that DD forces and then REDIRECT the forcing. A model that simply wrote a colour for whichever unknown node came next in index order would pass them both. So break the forcing instead of redirecting it:

   recolour w to a colour that ANOTHER known neighbour of v already holds.

Now the freed colour a is held by nobody, v's known neighbours cover only q-2 = 2 colours, v has TWO candidates, and DD does not emit v in this round at all.

The measurement moves ONE TOKEN TO THE LEFT: from the logits that predict v's COLOUR to the logits that predict WHICH NODE comes next. Does the model stop naming v?

```
   clean      ... BEGIN_ROUND  [v]  ->  c       DD emits v, and so should the model
   broken     ... BEGIN_ROUND  [ ? ]            v is no longer determined
```

PLACEBO, run on every single case: the same kind of one-token edit, but on a hint that is NOT a neighbour of v. v stays forced and DD's round is unchanged, so the model should still name v. That is the ceiling the broken rate is read against -- without it, a low rate would only show that edits perturb the node stream.

Pre-registered spoiler case: if the placebo collapses too, the test says nothing. It does not collapse, but it does not reach 1.000 either (0.90-0.96 across models), so a few points of every rate below are edit perturbation -- which is why all three models are read against it.

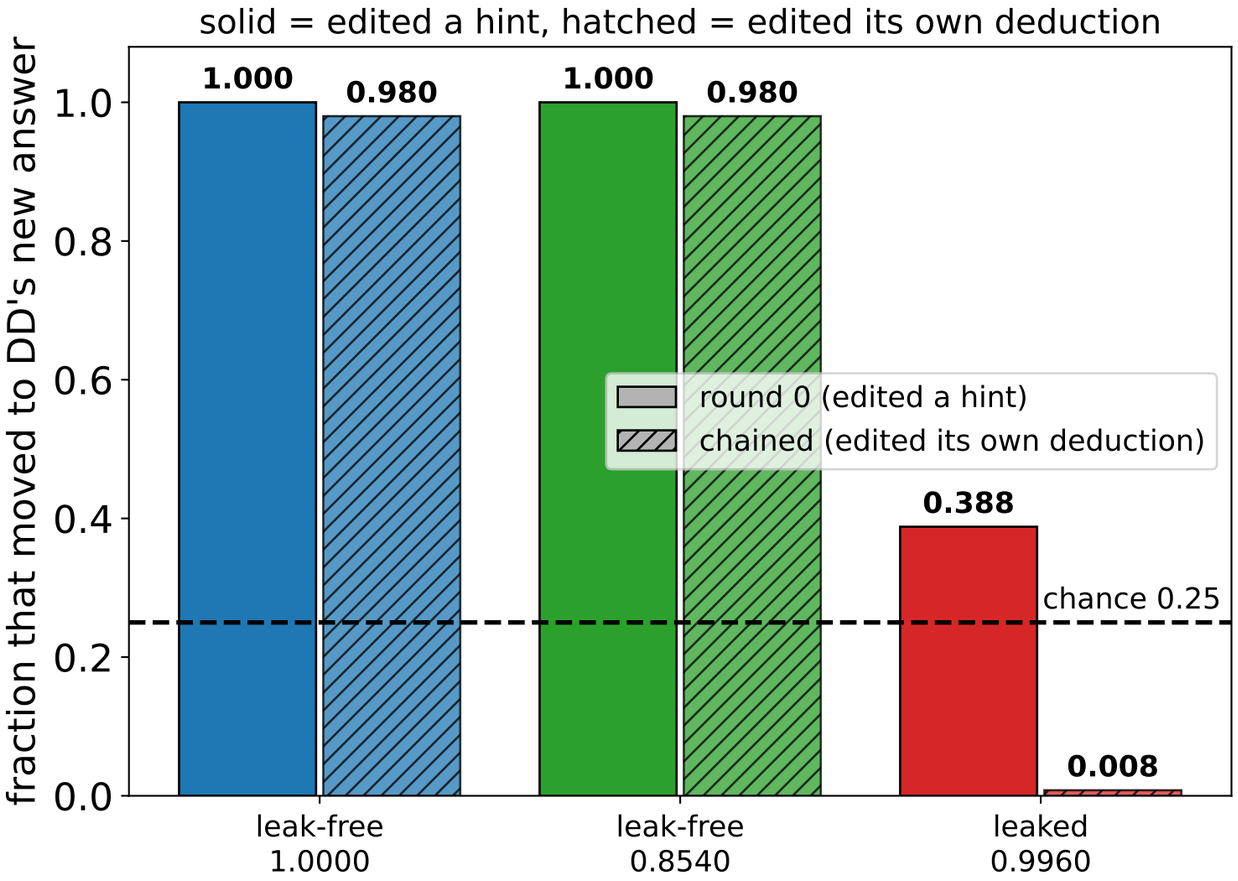
Every causal measurement

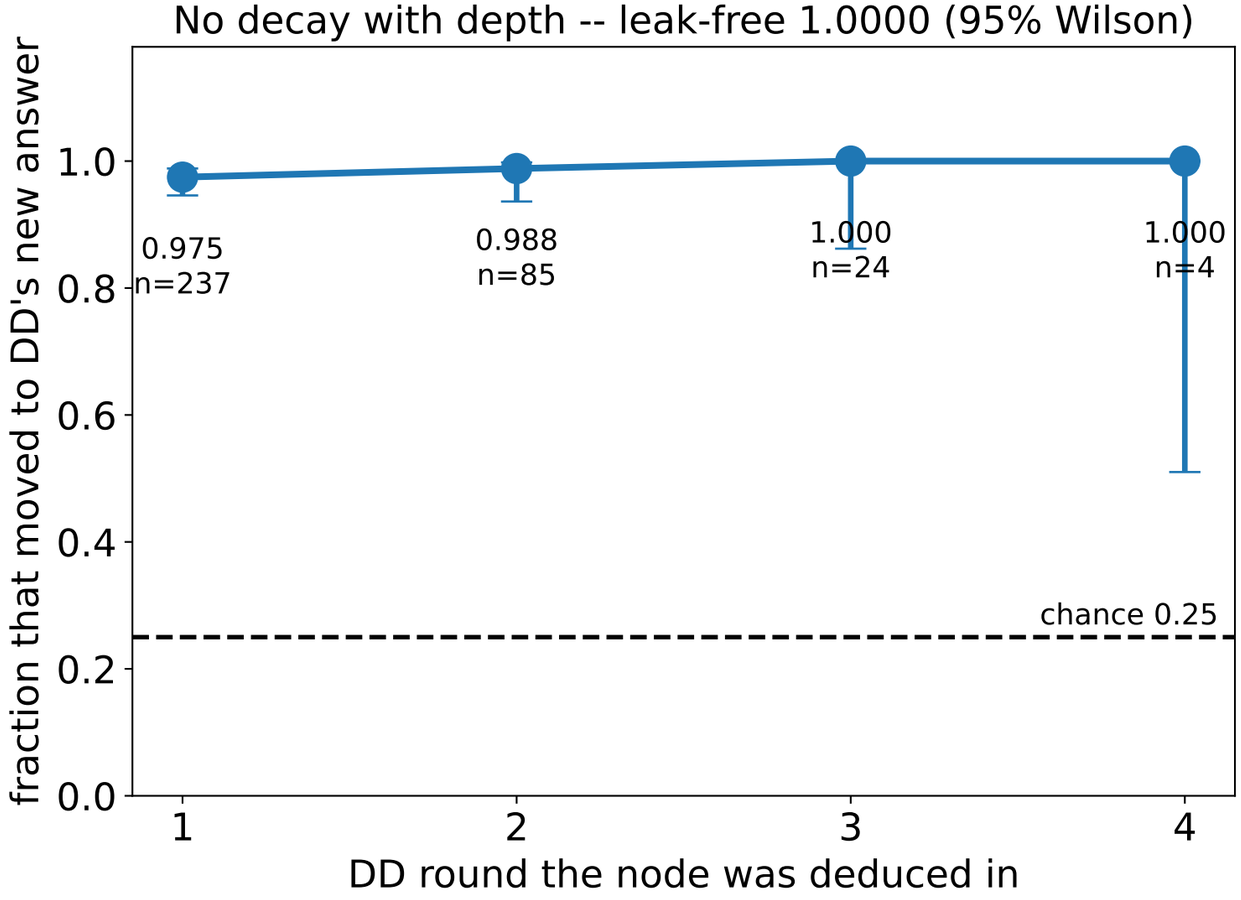
model	test	n	clean	moved to DD's new answer	kept its old answer	logit gap (rule) or placebo moved (guard)	verdict
leak-free 1.0000	round 0	350	1.000	**1.000**	--	-15.5 -> +15.7	uses the rule
leak-free 1.0000	chained	350	1.000	**0.980**	0.020	--	chains
leak-free 1.0000	withhold	350	1.000	**1.000**	0.000	placebo 0.060	guards
leak-free 0.8540	round 0	250	1.000	**1.000**	--	-15.0 -> +14.9	uses the rule
leak-free 0.8540	chained	250	1.000	**0.980**	0.008	--	chains
leak-free 0.8540	withhold	250	0.936	**0.824**	0.176	placebo 0.100	guards
leaked 0.9960	round 0	250	1.000	**0.388**	--	-14.5 -> -2.0	near chance
leaked 0.9960	chained	250	1.000	**0.008**	0.992	--	near chance
leaked 0.9960	withhold	250	1.000	**0.992**	0.008	placebo 0.036	guards

Every column is scored so that HIGH = followed DD, placebo included. A low 'moved' next to a high 'clean' means the model is right for some reason other than the elimination -- exactly the leaked model's profile, on rows 7 and 8.

On the withhold rows 'moved' = it stopped naming the node, 'kept its old answer' = it named it anyway, 'placebo moved' = how often the control edit also stopped it: the measurement's noise floor. 'clean' differs between a model's own rows because the conditions do -- the rule rows score the colour emitted for v, the withhold rows whether v is named at all. n is 350 or 250 by when each run was submitted, not by filtering; paired tests use the 250 shared cases.

The rule holds on given facts, and on facts it derived itself

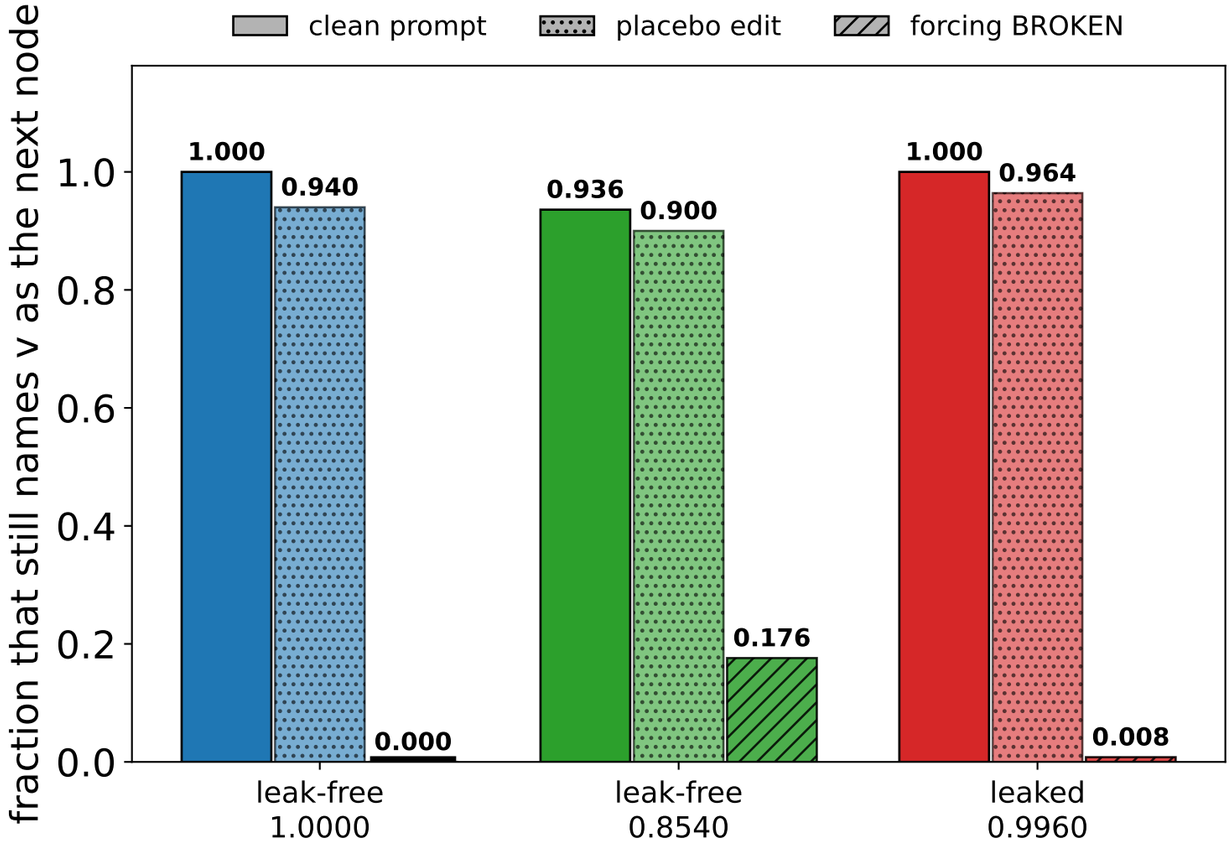


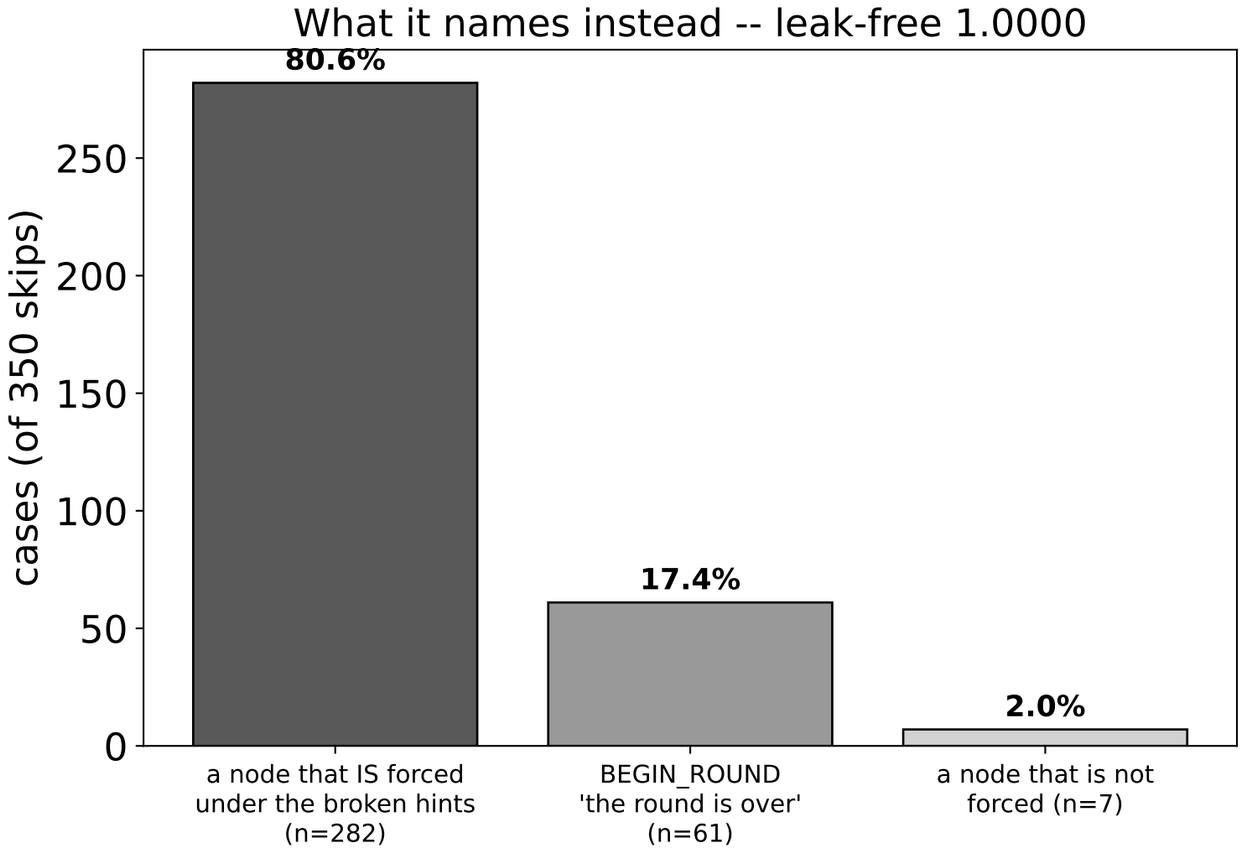


If the model chained one step but not three, round 1 would be high and the later rounds would fall off. It does not happen.

Right panel splits the chained test by how deep the edited deduction sat. Deep bins are small because deep chained forcings are rare in these instances, not because anything was filtered -- hence the intervals, wide at rounds 3-4 (n=24 and n=4) and still clear of chance. Rounds 5-6 are absent because no case in the sample had its binding neighbour deduced that late.

And it withholds: break the forcing and it stops naming the node

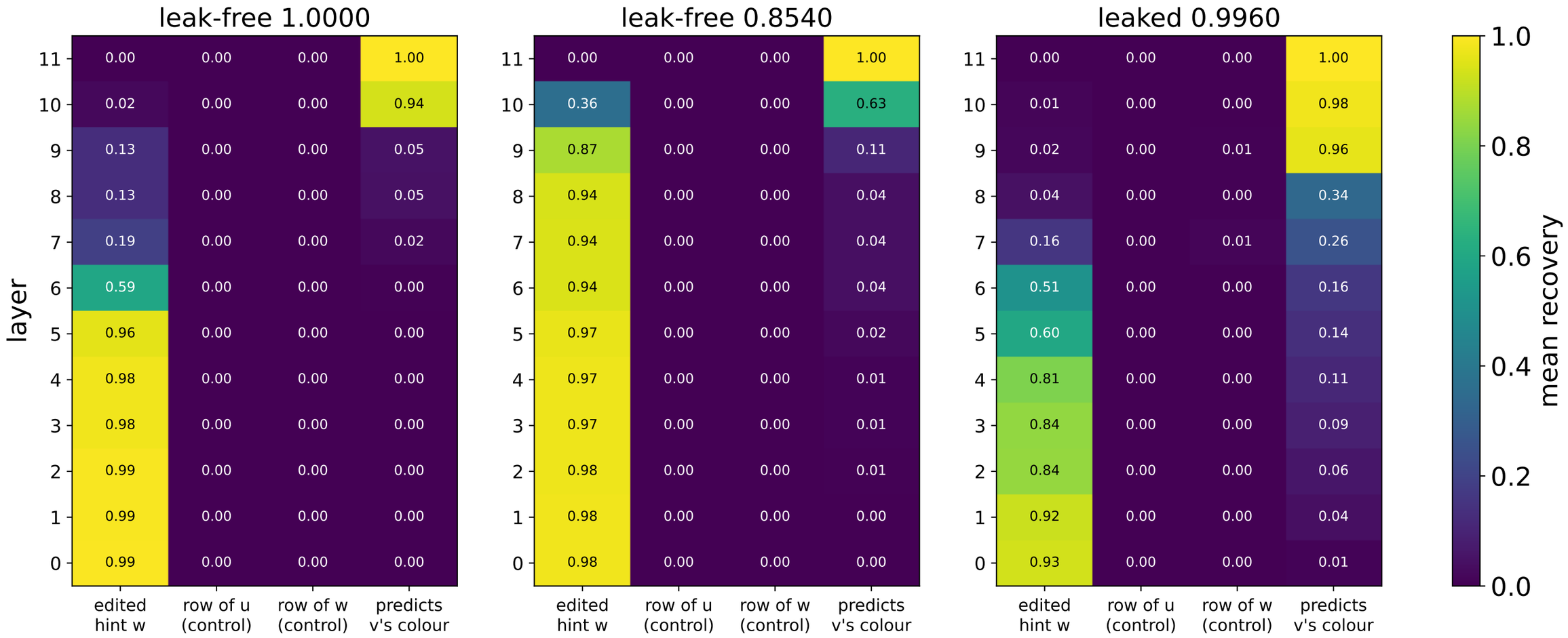




It does not just go quiet: of the 350 cases it withholds, it names another node that IS forced under the broken hints, or says BEGIN_ROUND -- which is DD's own answer when nothing else in the round is determined, not an error.

Same round-0 cases as the rule test, and the case lists are byte-identical across models, so 0.000 vs 0.176 is paired: 44 cases where the 0.8540 twin named v and the 1.0000 model did not, 0 the other way, McNemar exact p = 1.1e-13. Conditioning on cases the twin gets right on the clean prompt gives 0.188 (n=234). Its 0.176 guard gap is the same order as its 0.146 solve_rate gap -- different measurements on different samples, so a consistency check, not a decomposition.

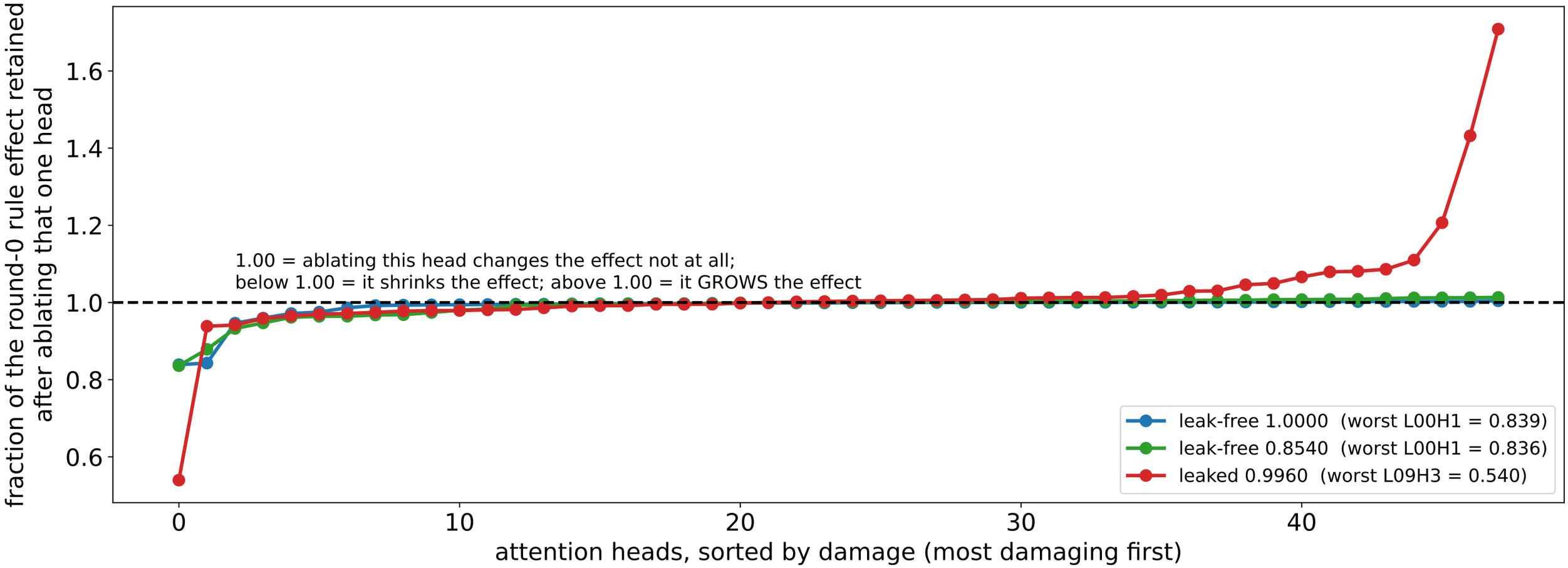
Where the recoloured fact travels (activation patching)



RECOVERY = copy one activation from the clean run into the corrupted run at that (layer, position) and measure how much of the flip it undoes: 1.00 = that one activation carries the whole edited fact, 0.00 = none of it. w is the neighbour the edit recolours, u is another known neighbour of v; both graph rows are controls, and 'predicts v's colour' is where the answer is read off.

Read upwards: the edited colour is held AT the edited hint through L5, handed off between L6 and L10, and has arrived at the prediction position by L10. The graph rows read 0.00 at every layer in every model -- the deduction never routes through the adjacency table. L11 at the prediction position is the built-in positive control and must read 1.00; it does. The 1.0000 model hands the fact off one to two layers earlier than its 0.8540 twin.

No single head carries the rule: removing the worst one leaves 84% of it



The leaked model's curve only looks steeper because its effect is near chance to begin with: a fraction of almost nothing.

Each point is one of the 48 heads (12 layers x 4), zeroed one at a time by writing its out_proj weight columns -- nn.MultiheadAttention takes the functional fast path, so module hooks on out_proj never fire, and an earlier version of this analysis silently reported 1.000 for all 7200 rows. Averaged over the same 350 (leak-free 1.0000) or 250 (the other two) round-0 cases as the rule test. Values above 1.00 mean ablating that head makes the model follow the edit MORE, which happens when a head is pulling toward the pre-edit answer.

The verdict, and what it is not

Working picture: sweep nodes in index order; emit one only when its known neighbours leave it a single colour, counting the ones it just derived. That is DD's propagation step.

WHAT IT DOES DO (causal)

* Applies DD's elimination rule: 1.000 on 350 one-token counterfactuals.

* Chains it off its OWN deductions: 0.980, no decay from round 1 to round 4.

* Guards it: break the forcing and it stops naming the node 1.000 of the time, while a placebo edit leaves it at 0.940.

* Its 0.854 twin guards only 0.824 -- a gap of 0.176, the same order as that twin's 0.146 solve_rate deficit, and the first test here to separate the two at all (p = 1e-13).

* Localised: edited hint -> L5, handoff L6-L10, prediction position by L10.

WHAT IT DOES NOT DO

* Choose the node ORDER by deduction. It emits nodes in ascending index order in 100% of rounds. That is also DD's own convention, so it proves nothing either way -- but the guard test does show that WHICH nodes it emits tracks determinacy.

* Route through the adjacency table: the graph rows recover 0.000 at every layer.

* The leaked twin never learned the rule (0.388, chance 0.250). Shown an edited deduction it keeps its own answer 99.2% of the time: it reads the adjacency-order leak, not the prefix. The earlier 'not running DD' conclusion on this project was always about THAT model.

LIMITATIONS: one checkpoint per model and no seed replicates, so every leak-free vs leaked contrast is also two separate training runs; one task configuration (N=24, degree 5, q=4); and the guard gap and the solve_rate gap are different measurements on different samples, so their agreement is a consistency check and not a decomposition.